Discretizing ODE for Quantum Computers

Adilet Akimshe

Advisor: Alejandro Javier Castro Second Reader: Adilet Otemissov

Department of Mathematics, Nazarbayev University

April 2026

Outline

1. Motivation
2. Background
3. Main Results
4. Conclusion

Quantum Computing: From Labs to Industry

Google Willow quantum processor:
surpassed classical supercomputers on benchmark tasks

D-Wave quantum annealer:
commercial quantum computing

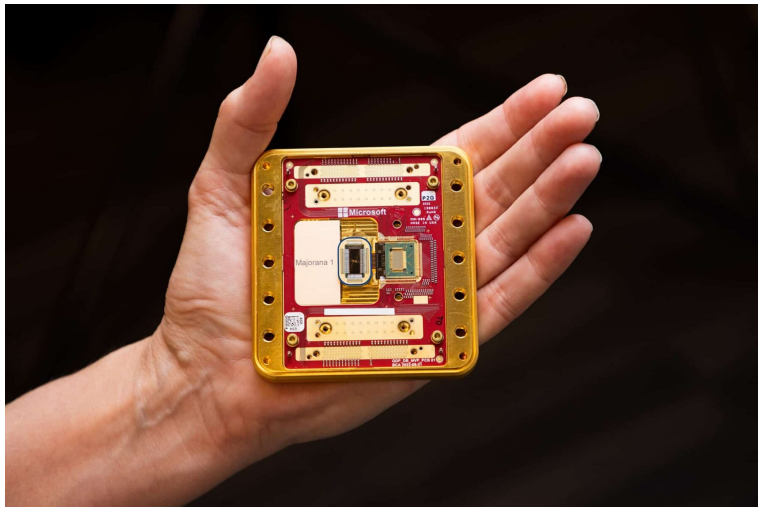
Quantum "arms race"

Google, IBM, Microsoft, IonQ, D-Wave — quantum hardware is advancing rapidly. Intermediary quantum algorithms are needed *now* to make use of near-term noisy devices before full fault-tolerance arrives. Whoever gets it first will be filthy rich.

Why Should Anyone Care?

The big picture

- Many scientific problems reduce to solving *linear systems* $A\mathbf{x} = \mathbf{b}$
- Classical cost: $O(N^3)$ (Gaussian) or $O(N^2)$ (iterative) — expensive for large N
- **HHL quantum algorithm**: $O(\log N)$ depth — exponential speedup in N, but requires tomorrow's fault-tolerant quantum computers
- **VQLS quantum algorithm**: runs on near-term devices, but has conditions on the matrix A (e.g. must be sparse in the Pauli basis)



Microsoft Majorana 1 chip (2025)

Simulation vs. Real Quantum Hardware

Classical (no quantum) simulation

- Runs on CPU/GPU — exact, no noise
- Memory requirement doubles with every qubit added

$\Rightarrow$ Feasible only up to $\sim$50 qubits

Our experiments use PennyLane's `lightning.qubit` *simulator — exact but expensive at* $N = 16$.

Real quantum hardware

- Operates on physical qubits — inherently noisy
- Gate errors, decoherence, measurement noise
- **NISQ era:** 50–1000 noisy qubits today

Why simulate at all?

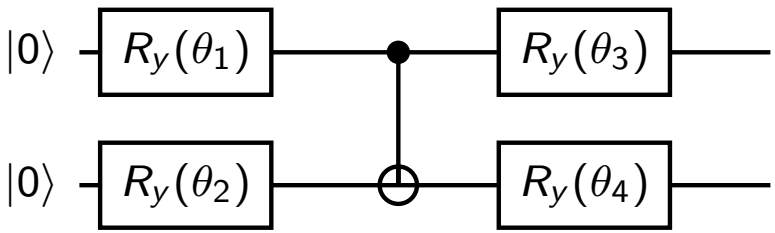
Simulation lets us *validate* algorithms cheaply before running on expensive quantum hardware.

Variational Quantum Linear Solver (VQLS)

Goal: solve $A|x\rangle = |b\rangle$ on a NISQ device.

Algorithm loop:

1. Prepare ansatz $|\psi(\boldsymbol{\theta})\rangle = V(\boldsymbol{\theta})|0\rangle$
2. Evaluate cost $C(\boldsymbol{\theta}) = 1 - \frac{|\langle b|A|\psi\rangle|^2}{\langle\psi|A^\dagger A|\psi\rangle}$
3. Update $\boldsymbol{\theta}$ by gradient descent
4. Repeat until $C \approx 0$



Hardware-efficient ansatz (2 qubits)

Analogy: supervised ML

ML	VQLS
Neural net	Quantum circuit
Weights $\boldsymbol{\theta}$	Gate angles
Loss	Cost C
SGD step	Gradient step
Forward pass	Circuit eval.

Two bottlenecks:

1. Circuit overhead: $O(L^2 n)$ circuits/step
2. Ansatz expressibility

Our work attacks **bottleneck 1**.

Pauli Strings

Pauli matrices:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Pauli strings — tensor products of these matrices. They form an orthogonal basis for *all* $2^n \times 2^n$ matrices. So any matrix A decomposes as

$$A = \sum_P c_P \, P, \qquad c_P = \tfrac{1}{2^n} \operatorname{Tr}(A\,P).$$

Pauli Strings — Concrete Example

Matrix (1 qubit, basis $\{I, X, Y, Z\}$):

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 1 \end{pmatrix}$$

Coefficients $c_P = \frac{1}{2}\,\mathrm{Tr}(AP)$:

$$c_I = 2, \quad c_X = 1, \quad c_Y = 0, \quad c_Z = 1.$$

Result — 3 nonzero Pauli strings:

$$A \;=\; 2I + X + Z$$

One can verify directly:

$$2I + X + Z = \begin{pmatrix} 3 & 1 \\ 1 & 1 \end{pmatrix} \checkmark$$

Why this matters for VQLS

One circuit per Pauli term: 3 circuits here. Y dropped out — fewer nonzero c_P means faster VQLS.

Collocation Discretization — Simple Example

Problem: $u'' = 0$, $u(0) = 0$, $u(1) = 1$. *Exact solution:* $u(t) = t$.

Spectral collocation ($N = 4$, monomial basis $\{1, t, t^2, t^3\}$):

1. Nodes: boundaries $t = 0, 1$ and interior (equally spaced) points $t = \frac{1}{3}, \frac{2}{3}$
2. Write $u(t) \approx c_0 + c_1 t + c_2 t^2 + c_3 t^3$
3. Enforce BCs at rows 0–1; enforce $u'' = 0$ at rows 2–3

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 4 \end{pmatrix}}_{A_{\text{mono}}} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

How many Pauli strings does A_{mono} decompose into?

A_{mono} Decomposes into All 16 Pauli Strings

Recall A_{mono} for $u'' = 0$ on $[0, 1]$:

$$A_{\text{mono}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 4 \end{pmatrix}$$

Using $c_P = \frac{1}{4}\,\mathrm{Tr}(A_{\text{mono}}\,P)$, *every* 2-qubit Pauli string has a nonzero coefficient:

P	c_P	P	c_P	P	c_P	P	c_P
II	$+2$	XI	$+\frac{1}{4}$	YI	$+\frac{1}{4}$	ZI	-1
IX	$+\frac{5}{4}$	XX	$+\frac{1}{4}$	YX	$+\frac{1}{4}$	ZX	$-\frac{3}{4}$
IY	$-\frac{1}{4}$	XY	$-\frac{1}{4}$	YY	$+\frac{1}{4}$	ZY	$-\frac{1}{4}$
IZ	$-\frac{1}{2}$	XZ	$-\frac{1}{4}$	YZ	$-\frac{1}{4}$	ZZ	$+\frac{1}{2}$

Cost for VQLS

L=16 terms $\Rightarrow$ **768** circuits/step.
Fewer c_P $\Rightarrow$ faster VQLS.

From Pauli Overhead to Basis Change

The cost bottleneck

VQLS costs $L^2(n+1)$ circuits/step, where $L =$ nonzero Pauli terms. With the monomial basis, A_{mono} gives $L = 16 \Rightarrow$ **768** circuits/step.

Key observation

The polynomial basis is a *free choice* — it changes A but **not the solution** u. What if we choose a different basis to get a different A with fewer nonzero Pauli terms?

Our idea

Better basis $\Rightarrow$ less Paulis $\Rightarrow$ fewer circuits $\Rightarrow$ faster VQLS.

The Basis-Change Identity

Key observation

The collocation matrix in *any* polynomial basis $\{\phi_j\}$ is

$$A_C = A_{\text{mono}}\, C^\top,$$

where C_{jk} = coefficient of t^k in ϕ_j. The physical solution u is **independent of the basis choice.**

Consequence:

- $C = I$ gives the monomial basis (Case A) with up to 4^n Pauli strings
- Any invertible C gives a valid discretization
- The Pauli coefficients of A_C are *linear* in the entries of C

$\Rightarrow$ **Finding the sparsest A_C is a linear program**

LP for Pauli minimization

$$\min_{C,\mathbf{t}} \sum_P t_P$$

$$\text{s.t. } |c_P(C)| \le t_P, \ \det C \neq 0$$

Each c_P is linear in C: **linear program**.

Global LP vs. Monic LP

Global LP — optimize all N^2 entries of C:

- Always finds $A^* = I_N$
- Only 1 Pauli string ($I^{\otimes n}$)
- Problem: $I_N |x\rangle = |b\rangle$ means $|x\rangle = |b\rangle$ — trivially classical, **no quantum advantage**

Monic LP — restrict C to lower-triangular, unit diagonal:

$$C = \begin{pmatrix} 1 & & & \\ & 1 & & \\ & * & 1 & \\ & * & * & 1 \end{pmatrix}$$

- Only $\frac{N(N-1)}{2}$ free variables
- $A^* \neq I_N$ — system stays non-trivial
- No extra normalization constraint needed
- Moderate but genuine Pauli reduction

The monic LP is the principled middle ground

Reduces circuit overhead substantially while preserving a meaningful quantum computation.

Results: Pauli String Counts

Experiment	N	Case A	Monic LP	Global LP
$u'' = 0$, $[0, 1]$	4	16	**6**	1
$u'' + u' + u = f$, $[-1, 1]$	4	16	**10**	1
$u'' + u' + u = f$, $[-1, 1]$ (8-pt)	8	64	**36**	1

Hadamard tests per gradient step:

	Case A	Monic LP	Global LP
$N = 4$	768	108	3
$N = 8$	16 384	5 184	4

Key takeaway

Monic LP:
7× speedup at N=4,
3.2× at N=8.
Problem stays non-trivial.

Timing Comparison Across System Sizes

Wall-clock time per VQLS gradient step on a CPU simulator (PennyLane, `lightning.qubit`).

Size	n	Case A (Monomial)		Monic LP		Global LP (I_N)	
		Paulis	s/step	Paulis	s/step	Paulis	s/step
4×4	2	16	17.2	6	1.3	1	0.05
8×8	3	64	$\sim$240	36	$\sim$75	1	0.11
16×16	4	256	$>$3000 *	112	$\sim$920 *	1	$\sim$0.3

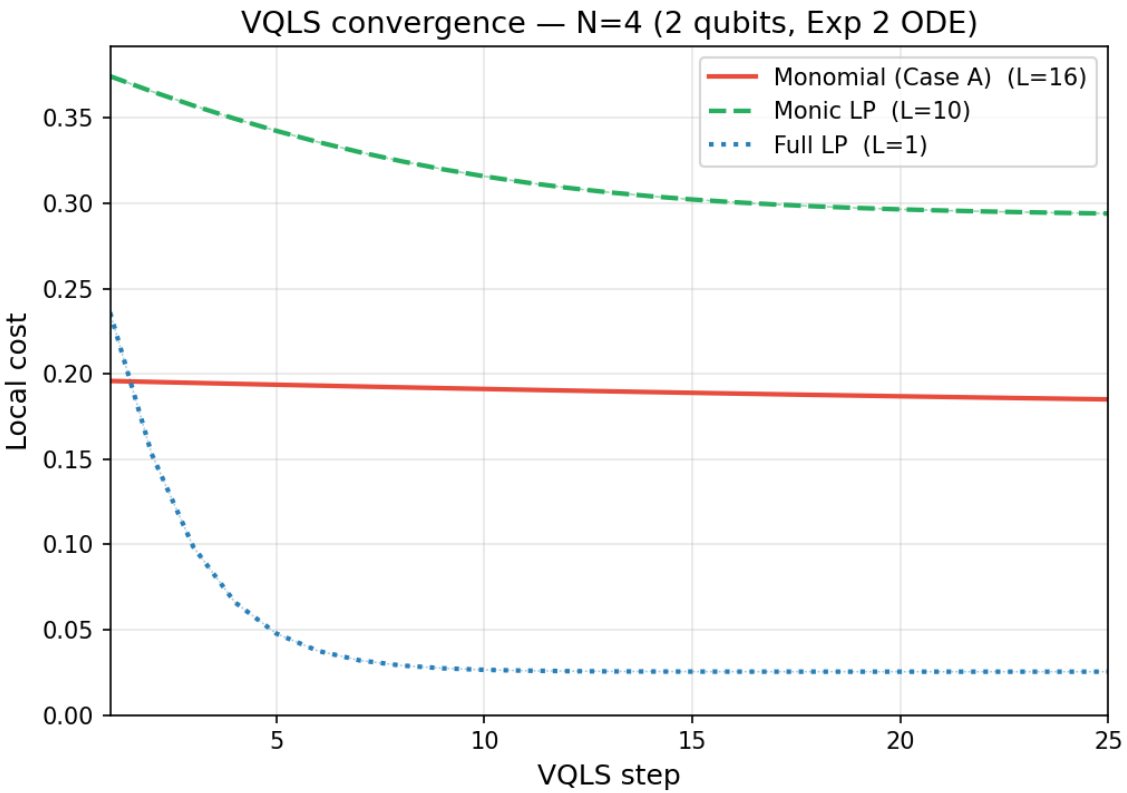
*Extrapolated from $N^4 \log N$ scaling; Case A at $N = 16$ is not feasible on CPU.

Scaling: Cost $\sim L^2 n \sim N^4 \log N$
Every doubling of N increases Case A cost by $\approx$16$\times$.

LP speedup grows with problem size

$N = 4$: $\approx$13$\times$ $N = 8$: $\approx$3.2$\times$
$N = 16$: $\approx$3.3$\times$ (projected)

VQLS Cost Convergence — $N = 4$ $(u'' + u' + u = f)$



Three bases, 25 gradient steps:

- **Case A** (L=16): plateaus at 0.185 immediately — too slow to escape
- **Monic LP** (L=10): starts higher, slowly descends, plateaus at 0.294
- **Full LP** (L=1): drops to 0.025 within 5 steps

Key message

All three plateau — *ansatz* is the binding limit. But Full LP reaches its plateau **237**$\times$ faster.

Convergence and Ansatz Limitation

Experiment 1 ($u'' = 0$, Global LP):

- Case A: cost 0.183 after 20 steps ($\approx$344 s). Stuck.
- Global LP: converges to $\approx$0 in **13 steps** (0.7 s)

Experiment 2 ($u'' + u' + u = f$, Global LP):

- Case A: stuck at 0.187 after 20 steps
- Global LP: drops to 0.025 in **4 steps** (0.25 s)
- Plateau at 0.025 = ansatz expressibility limit

Two independent bottlenecks

1. **Circuit overhead:** $O(L^2 n)$ Hadamard tests/step
   Fixed by LP preprocessing
2. **Ansatz expressibility:** circuit must represent $|x\rangle$
   Not affected by Pauli count — requires deeper ansatz

Both LP formulations share the same ansatz plateau — the overhead reduction is the sole benefit.

Conclusion

What we showed:

1. The collocation matrix is freely adjustable via basis change without altering the solution — enabling LP-based Pauli minimization
2. **Global LP** always finds $A^* = I_N$ (1 Pauli), but makes the problem classically trivial
3. **Monic LP** finds a non-trivial A^* with substantially fewer Paulis: $7\times$ speedup at $N = 4$, growing with N
4. Two independent VQLS bottlenecks: circuit overhead (cured by LP) and ansatz depth (separate issue)

Practical recommendation

Apply **monic LP preprocessing** before VQLS on near-term hardware. Costs milliseconds of classical LP time; saves orders-of-magnitude in circuit evaluations.

Future Work

Open problems

- **2D/3D PDEs:** joint basis-change LP over shared $B^\top$ — main open question
- **Node optimization:** Chebyshev/Legendre collocation nodes may reduce Paulis further
- **Deeper ansatz** to overcome expressibility plateau
- **Noise analysis** on real NISQ hardware

Key insight

In 1D, reducing Paulis removes quantum difficulty. **Multi-D PDEs** are the natural next target where quantum advantage may survive.

Thank you!

Questions?

Adilet Akimshe — NU, Dept. of Mathematics

Advisor: Alejandro Javier Castro Second Reader: Adilet Otemissov